

# **SIMULADOR DE ESCALONAMENTO**

**ALEX AKIO NISHIMURA JUNIOR – 081230007**  
**MARIA EDUARDA FERREIRA BIANCHINI – 081230001**  
**PEDRO HENRIQUE RODRIGUES DE ASSIS – 081230018**

## **DOCUMENTAÇÃO TÉCNICA**

**SÃO BERNARDO DO CAMPO**  
**2026**

# SUMÁRIO

|                                               |   |
|-----------------------------------------------|---|
| Sumário.....                                  | 1 |
| Lista de Tabelas .....                        | 2 |
| 1. Visão geral.....                           | 3 |
| 2. Separação entre política e mecanismo ..... | 3 |
| 3. Diagrama de módulos .....                  | 3 |
| 4. Estrutura de uma tarefa.....               | 4 |
| 5. Interface dos módulos.....                 | 5 |
| core.modelo .....                             | 5 |
| core.motor.....                               | 5 |
| core.validacao.....                           | 6 |
| core.geracao.....                             | 6 |
| gui.app .....                                 | 6 |
| gui.gantt.....                                | 6 |
| 6. Parâmetros configuráveis.....              | 7 |
| 7. Funcionamento interno .....                | 7 |
| 8. Convenções de simulação .....              | 8 |
| 9. Dependências e ambiente .....              | 9 |

## LISTA DE TABELAS

|                                         |   |
|-----------------------------------------|---|
| — <i>Estrutura de uma tarefa</i> .....  | 5 |
| — <i>Parâmetros configuráveis</i> ..... | 7 |

# DOCUMENTAÇÃO TÉCNICA

## 1. Visão geral

O simulador reproduz numericamente seis algoritmos de escalonamento de processos estudados em Sistemas Operacionais: FCFS (First-Come, First-Served), SJF (Shortest Job First), SRTF (Shortest Remaining Time First), Round-Robin, prioridade cooperativa e prioridade preemptiva. Além dos algoritmos de escalonamento, o simulador também considera a disputa por um recurso exclusivo R. Com isso, é possível observar situações de inversão de prioridade e testar os protocolos de herança e teto de prioridade. Também foi implementado o envelhecimento para reduzir problemas de inanição.

A entrada é um conjunto de tarefas, cada uma descrita por instante de ingresso, tempo de processamento, prioridade e, opcionalmente, uma seção crítica (uso do recurso R). A saída é, por tarefa e em média, o tempo de execução, o tempo de espera e o tempo até a primeira execução, além de um diagrama de tempo mostrando execução, trocas de contexto e bloqueios por recurso.

## 2. Separação entre política e mecanismo

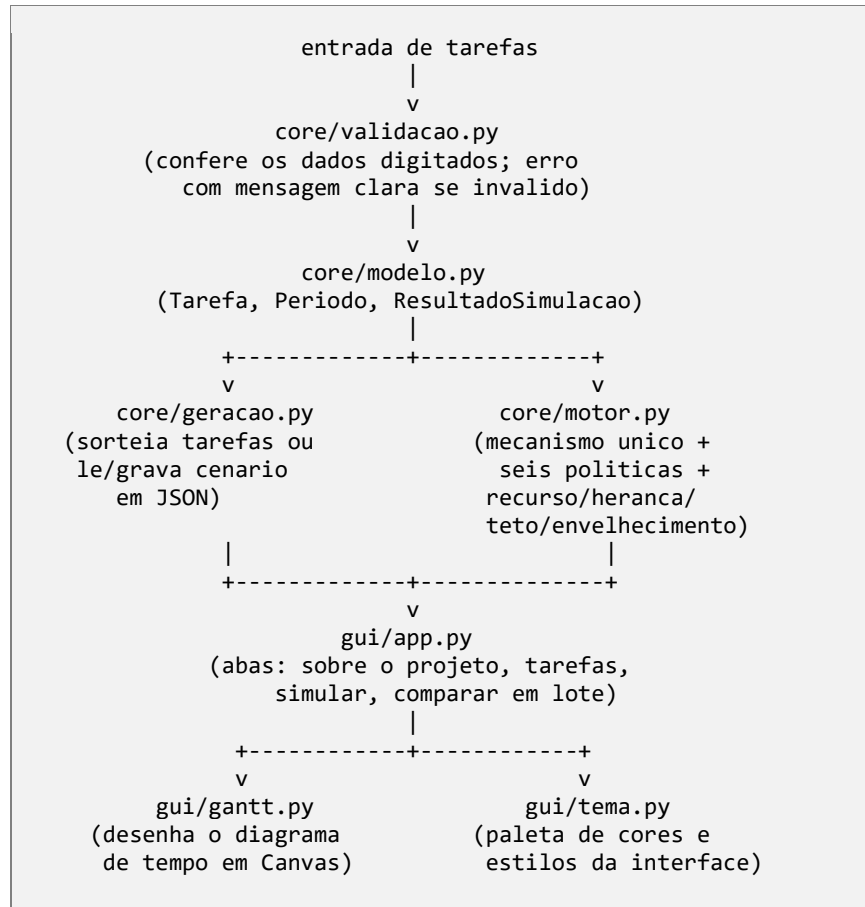
Uma das principais decisões de implementação do projeto foi separar política e mecanismo. O mecanismo é o conjunto de regras comuns a qualquer algoritmo de escalonamento: avançar o relógio, decidir quando uma tarefa é admitida na fila de prontas, cobrar a troca de contexto quando a tarefa em execução muda, e verificar quando uma tarefa termina. A política é a regra que decide, a cada instante, qual tarefa entre as prontas recebe o processador.

No código, o mecanismo vive em uma única função, `core.motor._executar`, usada por cinco dos seis algoritmos (FCFS, SJF, SRTF, prioridade cooperativa e prioridade preemptiva). A política de cada um desses cinco é uma função pequena — chamada de "chave de comparação" — que recebe uma tarefa e devolve um valor pelo qual as tarefas prontas são ordenadas; a tarefa de menor valor é a escolhida. O mecanismo não sabe qual critério está sendo usado; a política não sabe em que instante do relógio está. Isso é o que permite que FCFS, SJF, SRTF, prioridade cooperativa e prioridade preemptiva sejam, cada um, poucas linhas de código, apesar de produzirem comportamentos bem diferentes.

O Round-Robin é a exceção: seu mecanismo de despacho (fila circular com fatia de tempo fixa) é estruturalmente diferente do "escolher por critério", então ele tem sua própria função, `core.motor.round_robin`, que não reaproveita `_executar`.

## 3. Diagrama de módulos

O fluxo de dados segue sempre o mesmo sentido: a entrada (digitada ou sorteada) vira uma lista de objetos Tarefa; o motor consome essa lista e devolve um `ResultadoSimulacao`; a interface lê esse resultado e o apresenta como tabela de métricas e diagrama de tempo. Nenhum módulo de interface (gui/) contém lógica de escalonamento — toda decisão sobre "quem roda quando" está isolada em `core/motor.py`.



## 4. Estrutura de uma tarefa

| Campo           | Tipo                 | Função                                                         |
|-----------------|----------------------|----------------------------------------------------------------|
| id              | inteiro              | Identificador da tarefa                                        |
| chegada         | fração exata         | Instante em que a tarefa ingressa no sistema                   |
| tp              | fração exata         | Tempo de processamento total demandado                         |
| prioridade_base | inteiro              | Prioridade declarada; maior valor, prioridade mais alta        |
| sc_inicio       | fração exata ou nulo | Progresso próprio em que a tarefa solicita o recurso R         |
| sc_duracao      | fração exata ou nulo | Por quantas unidades de processamento a tarefa mantém R        |
| restante        | fração exata         | Quanto falta processar (decrece a cada unidade executada)      |
| progresso       | fração exata         | Quanto a própria tarefa já executou (localiza a seção crítica) |
| periodos        | lista de Período     | Histórico de execução, troca de contexto e bloqueio            |
| conclusao       | fração exata ou nulo | Instante em que a tarefa termina                               |

| Campo               | Tipo                 | Função                                                               |
|---------------------|----------------------|----------------------------------------------------------------------|
| ultimo_despacho     | fração exata ou nulo | Último instante em que a tarefa começou a executar (envelhecimento)  |
| tem_recurso         | booleano             | Se a tarefa está, neste momento, de posse de R                       |
| prioridade_override | inteiro ou nulo      | Prioridade elevada por herança/teto; nulo quando não há elevação     |
| bloqueios           | lista de tuplas      | Intervalos suspensa esperando R, classificados em direto ou inversão |

Tabela 1 — Estrutura de uma tarefa

Os campos chegada, tp, prioridade\_base, sc\_inicio e sc\_duracao são os dados de entrada, fixos durante toda a simulação. Os demais são estado de simulação, reiniciado pelo método Tarefa.reset() no começo de cada execução — necessário porque a mesma lista de tarefas costuma ser reaproveitada em várias simulações seguidas (por exemplo, na comparação em lote, que roda os seis algoritmos sobre o mesmo conjunto).

Os valores de tempo são armazenados utilizando fractions.Fraction. Essa escolha foi feita para evitar problemas de precisão que poderiam ocorrer com números de ponto flutuante, principalmente nas comparações realizadas durante a simulação.

## 5. Interface dos módulos

### core.modelo

- **Tarefa(id, chegada, tp, prioridade\_base, sc\_inicio=None, sc\_duracao=None)** — cria uma tarefa; converte os tempos para Fraction e inicializa o estado de simulação.
- **Tarefa.reset()** — restaura o estado de simulação, preservando os dados de entrada.
- **Tarefa.prioridade\_atual()** — devolve prioridade\_override se houver, senão prioridade\_base.
- **Tarefa.tempo\_execucao(), Tarefa.tempo\_espera(), Tarefa.primeira\_execucao()** — as três métricas de R3, calculadas a partir de conclusao, chegada, tp e periodos.
- **ResultadoSimulacao** — agrega a lista de tarefas já simuladas com o nome do algoritmo, ttc, tq (se houver) e o número de trocas de contexto; expõe media\_tt(), media\_tw(), media\_primeira\_execucao() e eficiencia().

### core.motor

- **fefs, sjf, srtf, prioridade\_cooperativa, prioridade\_preemptiva, round\_robin** — uma função por algoritmo; todas recebem a lista de Tarefa e o custo de troca de contexto (ttc); round\_robin recebe também o quantum (tq); prioridade\_cooperativa e prioridade\_preemptiva aceitam opcionalmente o passo de envelhecimento (alpha); prioridade\_preemptiva aceita opcionalmente o protocolo de correção (protocolo\_recurso: None, "heranca" ou "teto"). Todas devolvem um ResultadoSimulacao.

- **ALGORITMOS** — dicionário que mapeia a sigla do algoritmo à função correspondente, usado pela interface para despachar a simulação sem uma cadeia de if/elif.
- **ErroSimulacao** — exceção levantada quando a configuração impede a simulação de prosseguir (quantum menor ou igual ao custo de troca, ou impasse por recurso nunca liberado).

#### core.validacao

- **validar\_tarefa(...)** — confere os cinco campos de uma tarefa de uma vez; devolve os valores já convertidos ou levanta ValueError com mensagem descrevendo o campo e a regra violada.
- **validar\_quantum\_e\_ttc(...)** — confere quantum e custo de troca juntos, porque a regra depende dos dois ao mesmo tempo.

#### core.geracao

- **gerar\_tarefas(quantidade, chegada\_max, duracao\_max, prioridade\_max, permitir\_secao\_critica)** — sorteia uma lista de Tarefa dentro das faixas informadas: chegada entre 0 e chegada\_max, duração entre 1 e duracao\_max, prioridade entre 1 e prioridade\_max, todos inteiros. Quando permitir\_secao\_critica é verdadeiro, cada tarefa com duração maior ou igual a 2 tem 50% de chance de também receber uma seção crítica sorteada (início e duração dentro dos limites da própria duração).
- **salvar\_cenario(caminho, tarefas) / carregar\_cenario(caminho)** — serializam e desserializam uma lista de tarefas em JSON, preservando os tempos como texto para não perder precisão. Exemplo de um cenário salvo com duas tarefas, a segunda disputando o recurso R:

```
{
  "versao": 1,
  "tarefas": [
    { "id": 1, "chegada": "0", "tp": "5", "prioridade": 2 },
    { "id": 2, "chegada": "3", "tp": "2", "prioridade": 4, "sc_inicio": "1", "sc_duracao":
"1" }
  ]
}
```

#### gui.app

- **Aplicacao** — janela principal; mantém a lista de tarefas do cenário atual, compartilhada pelas abas.
- **AbaSobre** — tela informativa com a explicação do simulador e dos seis algoritmos.
- **AbaTarefas** — formulário de entrada, tabela do cenário atual, sorteio, salvar/carregar.
- **AbaSimulacao** — escolhe algoritmo e parâmetros, chama ALGORITMOS[sigla](...) e apresenta o resultado.
- **AbaComparacao** — repete a simulação dos seis algoritmos sobre vários cenários sorteados e mostra as médias.

#### gui.gantt

- **desenhar\_gantt(canvas, resultado)** — percorre os periodos e bloqueios de cada tarefa do ResultadoSimulacao e desenha o diagrama de tempo correspondente em um tkinter.Canvas.

## 6. Parâmetros configuráveis

| Parâmetro                        | Faixa válida                           | Valor padrão na interface | Onde se aplica                           |
|----------------------------------|----------------------------------------|---------------------------|------------------------------------------|
| Quantum (tq)                     | número positivo, maior que ttc         | 2                         | Round-Robin                              |
| Custo da troca de contexto (ttc) | número não negativo                    | 0                         | Todos os algoritmos                      |
| Passo do envelhecimento (alpha)  | número não negativo; opcional          | não definido              | Prioridade cooperativa e preemptiva      |
| Protocolo de correção            | nenhum / herança / teto                | nenhum                    | Prioridade preemptiva, com seção crítica |
| Instante de ingresso             | inteiro não negativo                   | —                         | Todas as tarefas                         |
| Tempo de processamento (tp)      | inteiro positivo                       | —                         | Todas as tarefas                         |
| Prioridade                       | inteiro positivo                       | —                         | Todas as tarefas                         |
| Início/duração da seção crítica  | inteiros não negativos; soma $\leq$ tp | não definida              | Tarefas que disputam o recurso R         |

*Tabela 2 — Parâmetros configuráveis*

Nenhum desses parâmetros está fixado no código-fonte; todos são informados pela interface, em tempo de execução, e recusados com mensagem de erro quando fora da faixa válida (ver core/validacao.py).

## 7. Funcionamento interno

A cada passo do laço principal (core.motor.\_executar), o motor:

1. Admite na fila de prontas as tarefas cujo instante de ingresso já chegou.
2. Se não houver tarefa em execução nem tarefa pronta, salta o relógio diretamente para o próximo instante de ingresso.
3. Se não houver tarefa em execução, escolhe a melhor entre as prontas, segundo a chave de comparação do algoritmo.
4. Se o algoritmo for preemptivo e já houver uma tarefa em execução, compara-a com a melhor tarefa pronta; se a pronta for estritamente melhor, a tarefa em execução volta à fila de prontas e a pronta passa a executar.



5. Se a tarefa que vai executar agora é diferente da que executou no passo anterior, cobra uma troca de contexto (de duração ttc) antes de prosseguir — inclusive no primeiro despacho da simulação.
6. Se o algoritmo trata recurso exclusivo (apenas a prioridade preemptiva) e a tarefa em execução atingiu, em seu próprio progresso, o início declarado de sua seção crítica: caso o recurso esteja livre, a tarefa o obtém (e, sob o protocolo de teto, assume imediatamente a prioridade de teto do recurso); caso esteja ocupado, a tarefa é suspensa — sai inteiramente do conjunto de prontas, independente de sua prioridade — e, sob o protocolo de herança, a tarefa detentora do recurso assume a maior prioridade entre a sua própria e a das tarefas suspensas aguardando por ele.
7. Executa a tarefa corrente por uma unidade de tempo, registrando o período de execução.
8. Se alguma tarefa está suspensa aguardando o recurso, registra esse intervalo como bloqueio direto (quando quem está executando é a própria detentora do recurso) ou como inversão de prioridade (quando quem está executando é uma terceira tarefa, que não usa o recurso).
9. Se a tarefa em execução concluiu sua seção crítica neste passo, libera o recurso, reverte sua prioridade para o valor original e promove à fila de prontas a tarefa suspensa de maior prioridade, se houver.
10. Se a tarefa em execução concluiu todo o seu processamento, registra o instante de conclusão e a remove da simulação.

Sob prioridade cooperativa e preemptiva com envelhecimento habilitado, a prioridade usada em toda comparação não é a prioridade base, mas a prioridade efetiva: a prioridade base somada ao passo de envelhecimento multiplicado pelo tempo decorrido desde o último despacho da tarefa (ou desde seu ingresso, se ela nunca executou). Essa prioridade efetiva é recalculada a cada comparação, nunca armazenada, e volta ao valor base assim que a tarefa recebe o processador.

Sob herança e teto, a prioridade usada em toda comparação é a prioridade atual da tarefa (prioridade\_override, quando definida, ou a prioridade base), e não a prioridade base diretamente — é essa indireção que permite às duas correções elevarem a prioridade de uma tarefa sem que o mecanismo de escolha precise saber que uma elevação está em curso.

O Round-Robin segue um laço próprio: a cada despacho, retira a tarefa da cabeça da fila circular; cobra troca de contexto se a tarefa despachada for diferente da anterior; concede uma fatia de tempo igual ao quantum menos o custo da troca, quando houve troca, ou igual ao quantum inteiro, quando não houve; se a tarefa não termina dentro da fatia, ela retorna ao final da fila, mas somente depois de nela também serem inseridas as tarefas que ingressaram exatamente naquele instante.

## 8. Convenções de simulação

Estas são as decisões de modelagem que determinam os números produzidos; alterar qualquer uma delas produz resultados diferentes para o mesmo conjunto de tarefas.

- **Unidade de tempo:** o relógio da simulação avança em unidades discretas de 1. Quantum e custo de troca podem ser fracionários, mas cada passo de execução do laço principal corresponde a exatamente uma unidade de trabalho.

- **Desempate:** quando duas ou mais tarefas empatam no critério principal do algoritmo, vence a de menor instante de ingresso; se ainda houver empate, vence a de menor identificador.
- **Prioridade:** valor maior significa prioridade mais alta.
- **Troca de contexto:** é cobrada sempre que a tarefa despachada é diferente da última que ocupou o processador — inclusive no primeiro despacho da simulação. No Round-Robin, o custo da troca é descontado da fatia concedida (a tarefa executa quantum - ttc unidades úteis quando há troca), nunca somado a ela; nos demais algoritmos, sem fatia fixa, a troca é um intervalo de tempo à parte, que precede a execução.
- **Retorno à fila no Round-Robin:** uma tarefa que esgota o quantum sem concluir retorna ao final da fila, mas depois das tarefas que ingressaram naquele mesmo instante.
- **Seção crítica:** medida no progresso próprio da tarefa, não no relógio da simulação — início 1 e duração 4 significam que a tarefa obtém o recurso após executar 1 unidade própria e o mantém até ter executado 5 unidades próprias.
- **Tempo de espera:**  $tw = tt - tp$ ; engloba fila, suspensão por recurso e troca de contexto.
- **Teto do recurso:** calculado uma única vez, no início da simulação, como a maior prioridade entre todas as tarefas que declaram uso do recurso — mesmo que a disputa por ele não chegue a ocorrer de fato.
- **Herança e teto não se combinam:** são protocolos alternativos; a simulação usa no máximo um dos dois por vez.
- **Envelhecimento:** conta o tempo decorrido desde o último despacho da tarefa, ou desde seu ingresso, caso ela ainda não tenha executado.
- **Relógio ocioso:** se não há tarefa em execução nem tarefa pronta, o relógio salta diretamente para o instante de ingresso da próxima tarefa, em vez de avançar unidade por unidade sem trabalho útil.

## 9. Dependências e ambiente

O programa roda em Python 3.10 ou superior e não depende de nenhuma biblioteca externa em tempo de execução — usa apenas módulos da biblioteca padrão (tkinter para a interface gráfica, fractions para aritmética exata, json para salvar/carregar cenários, random para o sorteio).

O executável de distribuição é gerado com PyInstaller (versão 6 ou superior, listada em requirements.txt), a partir da receita em SimuladorEscalonamento.spec, com o comando:

```
python3 -m PyInstaller SimuladorEscalonamento.spec
```

O PyInstaller é uma ferramenta de build, não uma dependência do programa em si: o resultado é um único executável autocontido, que embute o interpretador Python e não requer nenhuma instalação na máquina de quem o executa.